Library Management System

A Desktop Application using Python (Tkinter & SQLite)

PROJECT REPORT

Submitted by	Sagar Mishra
Course	Introduction to Problem Solving and Programming
Institution	VIT Bhopal
Project Name	Library Management System
Date	28 September 2026

Table of Contents

- 1. Introduction
- 2. Objectives
- 3. System Requirements
- 4. System Design
- 5. Database Design
- 6. Module Description
- 7. Key Code Snippets
- 8. Working / Features Summary
- 9. Testing
- 10. Limitations
- 11. Future Scope
- 12. Conclusion

1. Introduction

The Library Management System is a desktop application built to simplify day-to-day record-keeping in a small library or reading room. Traditionally, tracking which books are available, which are issued, and to whom, is done manually using registers, which is slow and prone to error. This project replaces that manual process with a simple graphical application that stores all book records in a database and lets a librarian add, search, issue, return, and delete book entries through an easy-to-use interface.

The application is built using Python's Tkinter library for the graphical user interface and SQLite for persistent data storage. SQLite was chosen because it is lightweight, requires no separate server installation, and stores the entire database in a single local file, making the application fully self-contained and easy to run on any machine with Python installed.

2. Objectives

- To design a simple, intuitive interface for managing a library's book records.
- To allow adding new books with title, author, and status details.
- To provide a search feature that filters books by title or author.
- To track the issue and return status of every book in real time.
- To allow safe deletion of book records with a confirmation prompt.
- To persist all data using a local SQLite database so records survive between sessions.

3. System Requirements

3.1 Software Requirements

Component	Requirement
Operating System	Windows / macOS / Linux
Programming Language	Python 3.8 or above
GUI Library	Tkinter (built into Python standard library)
Database	SQLite 3 (built into Python standard library)
IDE / Editor	Any Python-compatible editor (VS Code, PyCharm, etc.)

3.2 Hardware Requirements

Component	Minimum Requirement
Processor	1 GHz or above
RAM	512 MB or above
Storage	50 MB free disk space

4. System Design

The application follows a simple two-layer design: a presentation layer built with Tkinter widgets, and a data layer handled by SQLite through Python's built-in sqlite3 module. There is no separate business-logic layer; each GUI action (button click) directly calls a Python function that performs validation, executes the relevant SQL statement, and refreshes the on-screen list.

4.1 Application Flow

1. The application starts and connects to (or creates) the library.db SQLite file.
2. The books table is created automatically if it does not already exist.
3. All existing records are loaded into the listbox on startup.
4. The user adds, searches, issues, returns, or deletes books using the on-screen controls.
5. Every action commits changes to the database immediately and refreshes the visible list.
6. The database connection closes when the application window is closed.

5. Database Design

The system uses a single SQLite table named **books**, which stores all information needed to track a book's identity and current status.

Field	Type	Description
id	INTEGER (Primary Key, Autoincrement	)Unique identifier for each book
title	TEXT (Not Null)	Title of the book
author	TEXT (Not Null)	Author of the book
status	TEXT (Default: 'Available')	Current status - Available or Issued

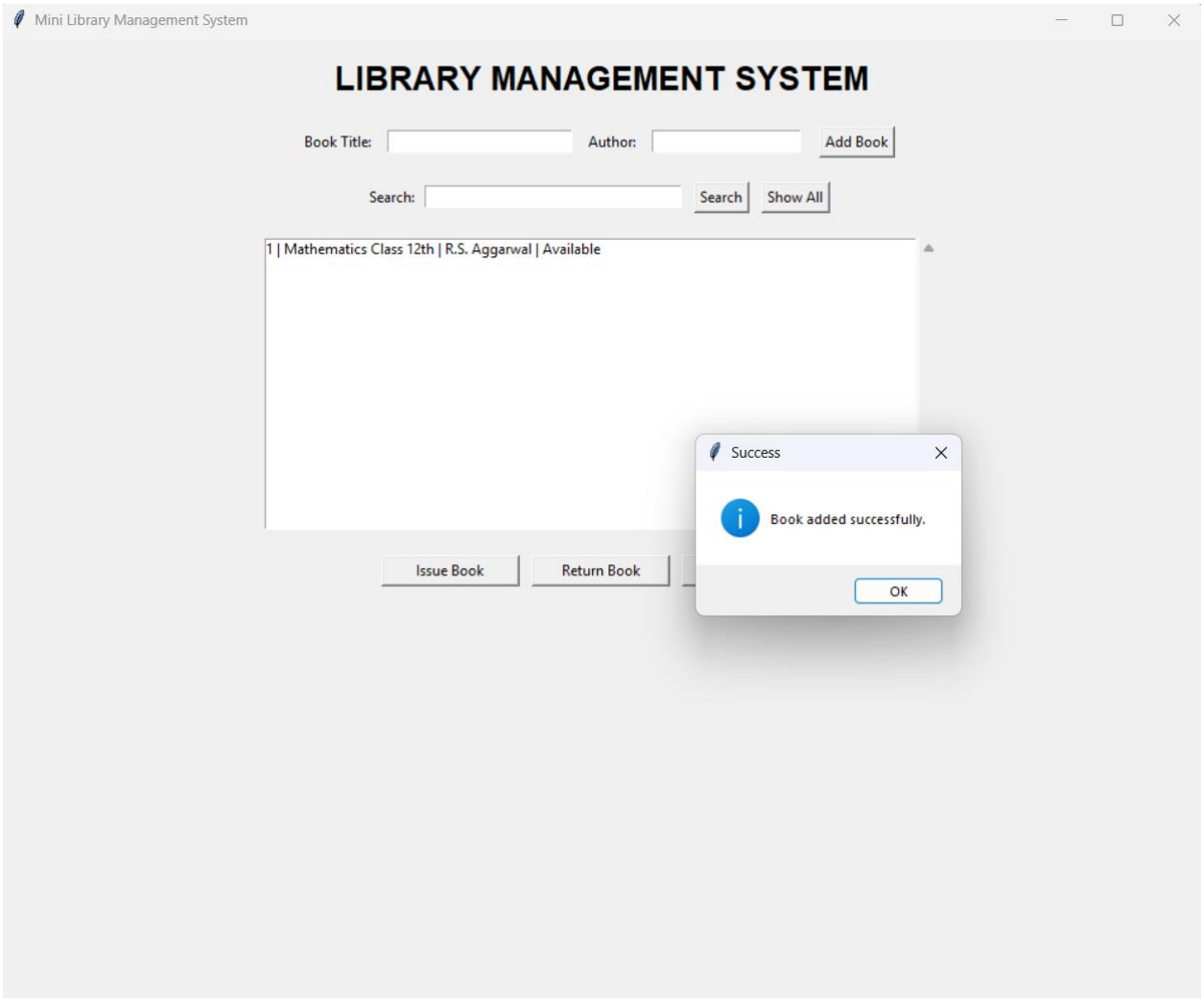
6. Module Description

refresh_books()

Clears and reloads the listbox with the current contents of the books table. Called after every add, issue, return, or delete operation, and once at startup.

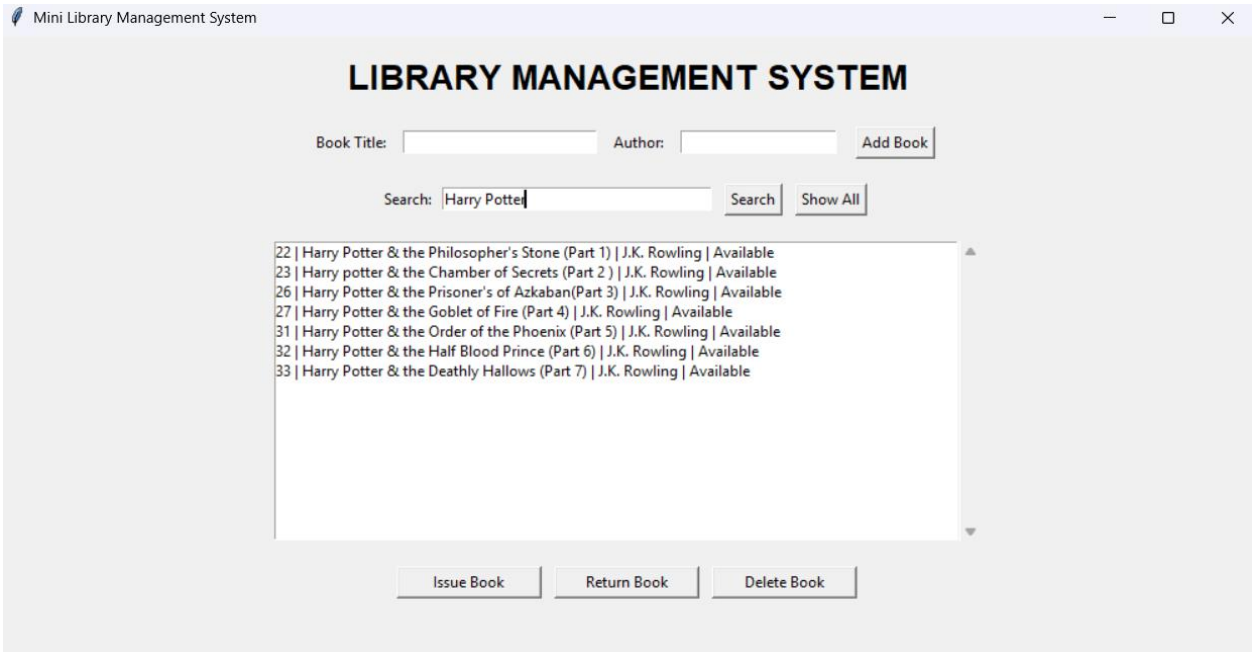
add_book()

Reads the title and author fields, validates that neither is empty, inserts a new row into the database, clears the input fields, and refreshes the list.



search_book()

Filters the listbox to show only books whose title or author matches the text typed into the search box, using a SQL LIKE query.

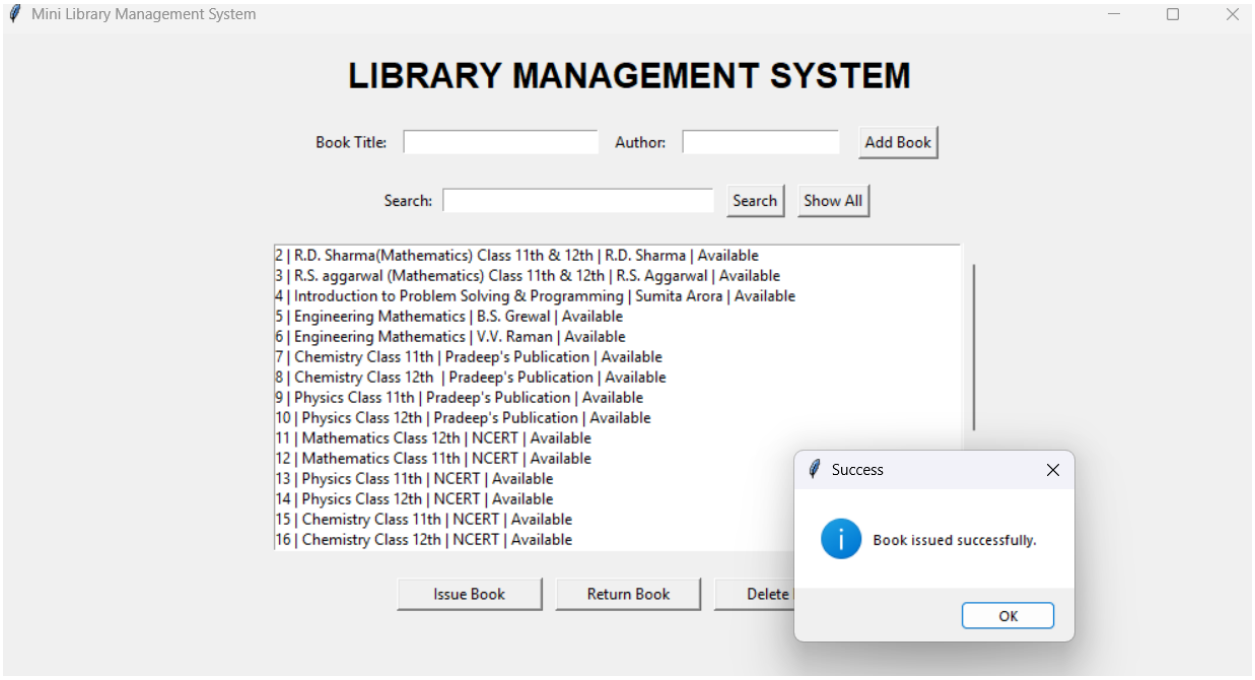


get_selected_book_id()

Reads the currently highlighted row in the listbox and extracts its id, or shows a warning if nothing is selected.

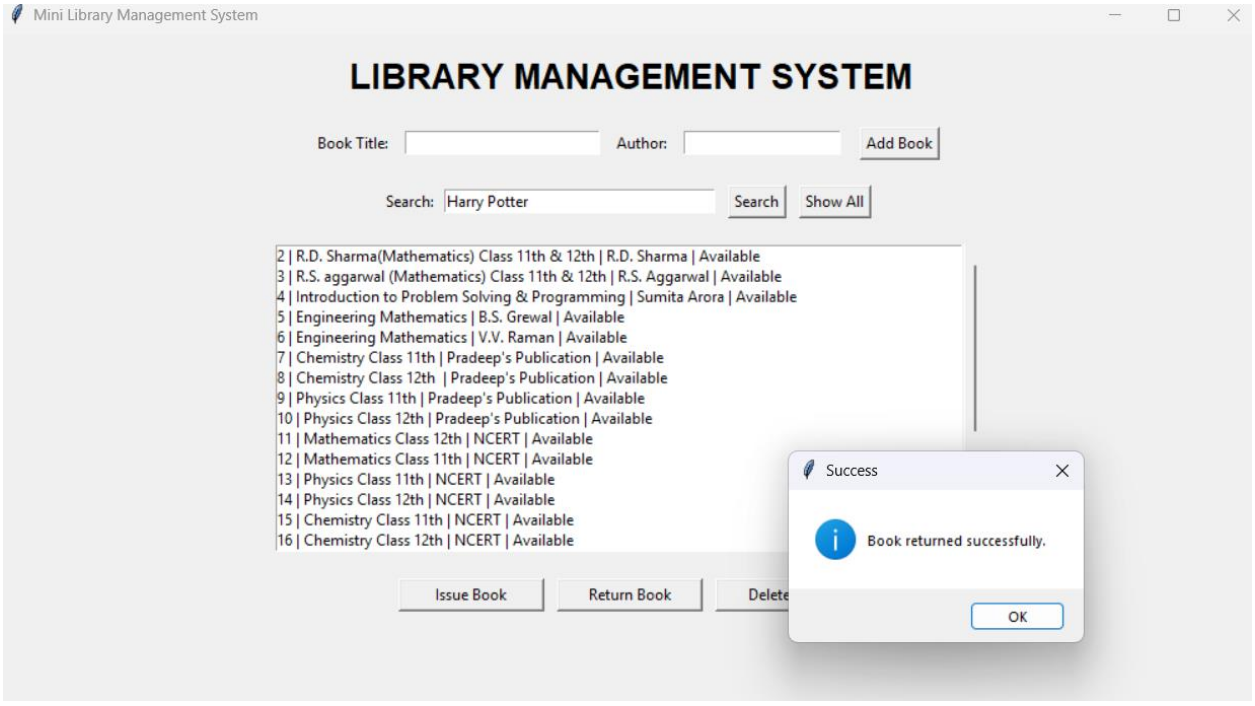
issue_book()

Marks the selected book's status as 'Issued', after checking it is not already issued.



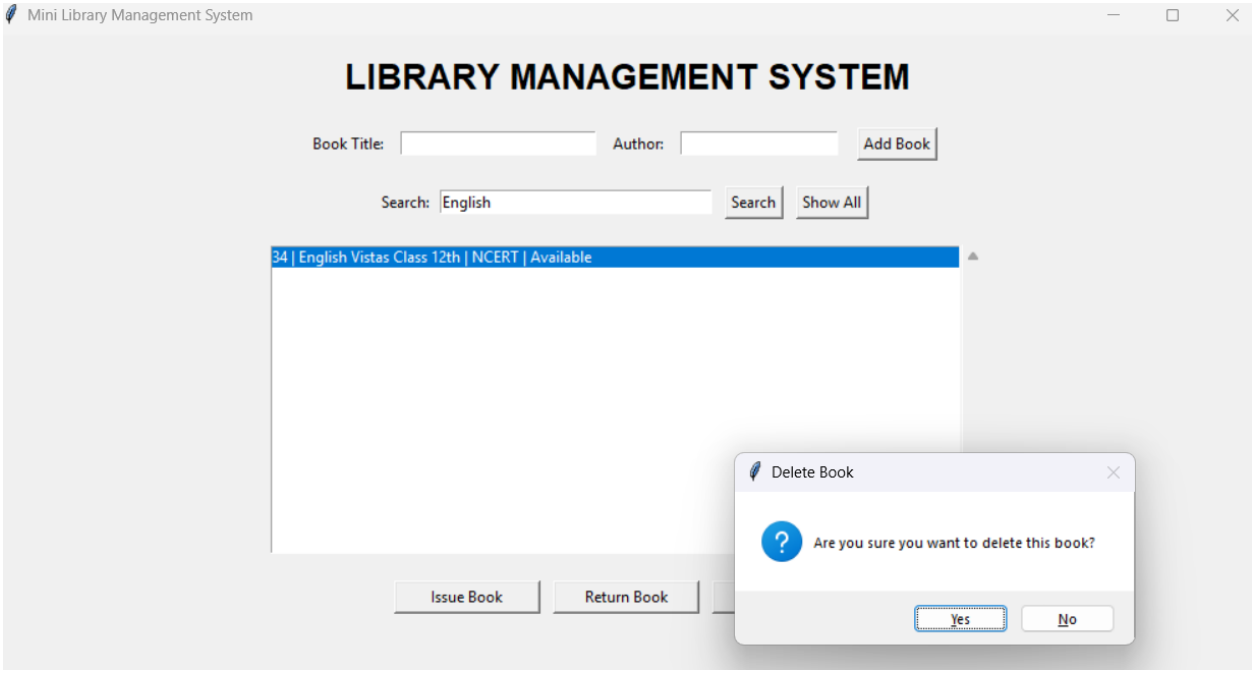
return_book()

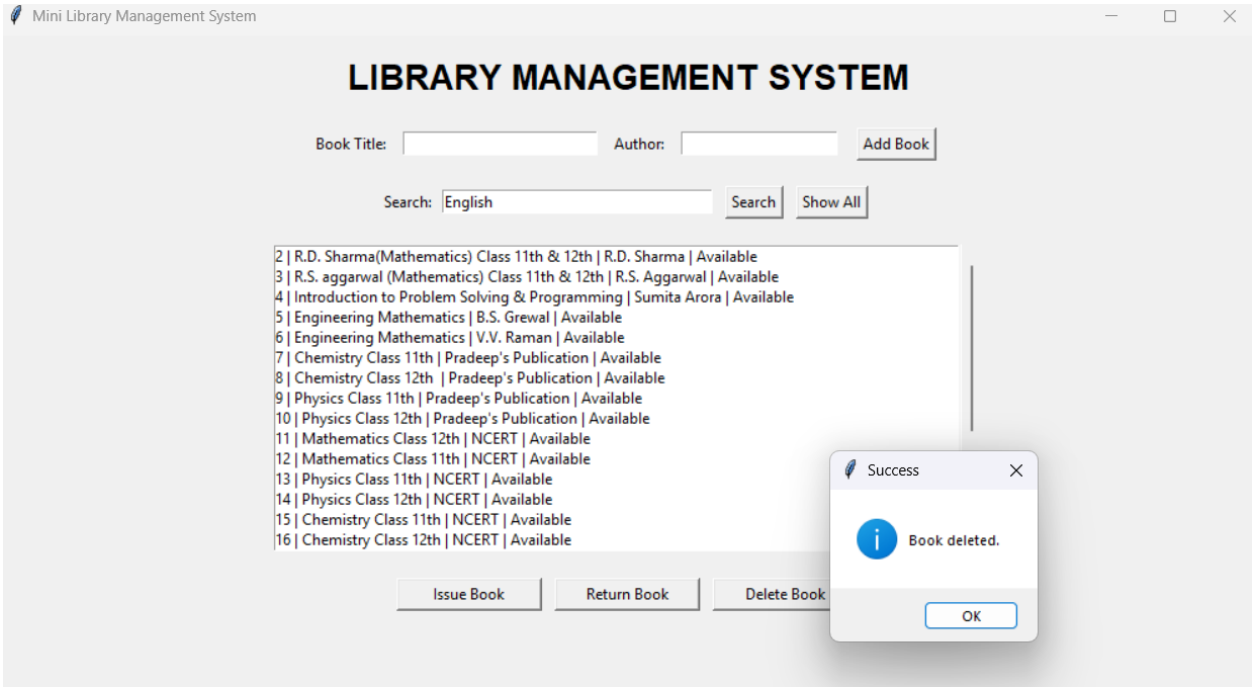
Marks the selected book's status as 'Available', after checking it is not already available.



delete_book()

Asks for confirmation, then permanently removes the selected book record from the database.


Mini Library Management System
LIBRARY MANAGEMENT SYSTEM
Book Title:
Author:
Add Book
Search:
English
Search
Show All
34 | English Vistas Class 12th | NCERT | Available
Issue Book
Return Book
Delete Book
Are you sure you want to delete this book?
Yes
No


Mini Library Management System
LIBRARY MANAGEMENT SYSTEM
Book Title:
Author:
Add Book
Search:
English
Search
Show All
2 | R.D. Sharma(Mathematics) Class 11th & 12th | R.D. Sharma | Available
3 | R.S. aggarwal (Mathematics) Class 11th & 12th | R.S. Aggarwal | Available
4 | Introduction to Problem Solving & Programming | Sumita Arora | Available
5 | Engineering Mathematics | B.S. Grewal | Available
6 | Engineering Mathematics | V.V. Raman | Available
7 | Chemistry Class 11th | Pradeep's Publication | Available
8 | Chemistry Class 12th | Pradeep's Publication | Available
9 | Physics Class 11th | Pradeep's Publication | Available
10 | Physics Class 12th | Pradeep's Publication | Available
11 | Mathematics Class 12th | NCERT | Available
12 | Mathematics Class 11th | NCERT | Available
13 | Physics Class 11th | NCERT | Available
14 | Physics Class 12th | NCERT | Available
15 | Chemistry Class 11th | NCERT | Available
16 | Chemistry Class 12th | NCERT | Available
Issue Book
Return Book
Delete Book
Success
Book deleted.
OK

7. Key Code Snippets

7.1 Table Creation

```
CREATE TABLE IF NOT EXISTS books (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  author TEXT NOT NULL,
  status TEXT DEFAULT 'Available'
)
```

7.2 Adding a Book

```
cursor.execute(
  "INSERT INTO books (title, author) VALUES (?, ?)",
  (title, author)
)
conn.commit()
```

7.3 Searching Books

```
cursor.execute(
  "SELECT * FROM books WHERE title LIKE ? OR author LIKE ?",
  (f"%{search}%", f"%{search}%")
)
```

7.4 Issuing a Book

```
cursor.execute("SELECT status FROM books WHERE id = ?", (book_id,))
result = cursor.fetchone()

if result[0] == "Issued":
  messagebox.showwarning("Warning", "This book is already issued.")
  return

cursor.execute("UPDATE books SET status = ? WHERE id = ?", ("Issued", book_id))
conn.commit()
```

8. Working / Features Summary

- Add Book - registers a new book with title and author; status defaults to Available.
- Search - filters the visible list live by title or author keyword.
- Show All - resets the list view to display every record in the database.
- Issue Book - changes a selected book's status to Issued, with a duplicate-issue check.
- Return Book - changes a selected book's status back to Available.
- Delete Book - removes a record permanently, after a Yes/No confirmation dialog.

9. Testing

The application was tested manually by performing each operation and verifying the resulting state in both the interface and the underlying database file.

Test Case	Expected Result	Status
Add book with empty title/author	Warning dialog shown, no record inserted	Pass

Add book with valid data	Record inserted and visible in list	Pass
Search with matching keyword	Only matching records displayed	Pass
Search with no matches	Empty list displayed	Pass
Issue an already-issued book	Warning dialog shown, no change made	Pass
Issue an available book	Status updates to Issued	Pass
Return an already-available book	Warning dialog shown, no change made	Pass
Delete without selecting a row	Warning dialog shown	Pass
Delete with confirmation	Record removed from list and database	Pass

10. Limitations

- No user authentication - anyone with access to the application can modify records.
- No member/borrower tracking - the system records that a book is issued, but not to whom.
- No due dates or fine calculation for overdue books.
- Single-table design; no support for multiple copies of the same title.

11. Future Scope

- Add a members/borrowers table to track who has issued each book, with contact details.
- Add due dates and automatic fine calculation for late returns.
- Introduce login-based access control for librarians and administrators.
- Support multiple copies of the same book title with individual copy tracking.
- Export reports (e.g., issued books, overdue books) to PDF or Excel.

12. Conclusion

The Library Management System successfully demonstrates how a lightweight desktop application can replace manual, paper-based tracking of library books. Using Python's Tkinter and SQLite, the project achieves a functional, self-contained tool that supports the core operations any small library needs: adding, searching, issuing, returning, and deleting book records. While the current version is limited to single-user, single-table record keeping, its modular function-based structure makes it straightforward to extend with member tracking, authentication, and reporting features in the future.